TECNOLÓGICO NACIONAL DE MÉXICO

TECNM

Instituto Tecnológico de Culiacán

Ingeniería en Sistemas Computacionales

Ciencia de Datos

08:00 – 09:00

Prof. Zuriel Dathan Mora Félix

Cristian Roberto Medina González 23170234

Yael Aram Lugo Ojeda 23170214

Unidad 2

Análisis exploratorio y limpieza de datos climáticos con Python

REPORTE DESCRIPTIVO DEL PROYECTO

Análisis exploratorio y limpieza de datos climáticos con Python

Estación meteorológica Acatitán · San Ignacio, Sinaloa

Documento de apoyo para describir los objetivos, el código, los procedimientos y los resultados obtenidos durante la exploración y limpieza de una base de datos climatológica.

Información del proyecto	Descripción
Fuente de datos	CONAGUA-DGE · Base de datos climatológica, estación 25001
Archivo de entrada	25001.txt
Entorno de trabajo	Python y notebook de Jupyter
Periodo de los registros	01/01/1961 al 31/07/2018
Producto del análisis	Datos explorados, valores faltantes identificados y reemplazados con la media

CONTENIDO

Descripción · Datos y objetivos · Funciones del programa · Resultados · Interpretación · Conclusiones · Código utilizado

1. Descripción del proyecto

Este proyecto tiene como finalidad explorar y preparar para su análisis una base de datos climatológica de la estación Acatitán, ubicada en el municipio de San Ignacio, Sinaloa. Los registros proceden de un archivo TXT proporcionado por CONAGUA e incluyen observaciones diarias de precipitación, evaporación, temperatura máxima y temperatura mínima.

Se utilizó Python para cargar el archivo, examinar su estructura, localizar valores faltantes, contar los registros incompletos y sustituir los datos ausentes por el promedio de cada variable. Posteriormente se compararon las estadísticas y algunas distribuciones antes y después de la limpieza. Todo el procedimiento se documentó en un notebook.

2. Objetivos

Objetivo general. Aplicar técnicas básicas de exploración, detección y tratamiento de datos faltantes a un conjunto de registros climatológicos, utilizando Python y Pandas.

- Cargar correctamente el archivo TXT y organizarlo en un DataFrame.
- Identificar y contar los datos nulos tanto por fecha como por columna.
- Reemplazar los datos faltantes por la media de cada variable numérica.
- Comparar los resúmenes estadísticos y las gráficas antes y después de la limpieza.

3. Características de los datos

El archivo 25001.txt contiene un encabezado descriptivo de la estación, seguido de los registros diarios. La lectura omite las primeras 19 líneas, interpreta los espacios como separadores y reconoce la palabra «Nulo» como un valor faltante. La línea final compuesta por guiones se descarta dentro de Python, sin cambiar el archivo original.

olumna	Qué representa	Unidad
ECHA	Día de observación	dd/mm/aaaa
RECIP	Precipitación diaria	mm
VAP	Evaporación diaria	mm
MAX	Temperatura máxima	°C
MIN	Temperatura mínima	°C

Conjunto de datos utilizado

Tras descartar la línea final no válida, se obtienen 19,574 registros con cinco columnas; cuatro de ellas corresponden a variables numéricas.

4. Herramientas empleadas

Pandas se utilizó para leer el archivo y manipular los DataFrames, identificar valores ausentes y calcular estadísticas. Matplotlib permitió construir histogramas y un diagrama de caja (boxplot) para comparar visualmente los datos originales con los completados.

5. Procedimiento y descripción del código

5.1 Carga y preparación: cargarCSV(ruta)

La función establece los nombres de las columnas y lee el archivo mediante pd.read_csv(). El parámetro sep=r"\s+" identifica separaciones por uno o más espacios; skiprows=19 omite el encabezado; na_values=["Nulo"] convierte esos textos en valores NaN; y encoding="latin-1" permite interpretar el archivo. Antes de devolver el DataFrame, comprueba si la última entrada en FECHA consta únicamente de guiones y, en ese caso, excluye esa última fila.

```
if not df.empty and str(df.iloc[-1]['FECHA']).strip('-') == '':
    df = df.iloc[:-1].copy()
```

Esta eliminación se realiza en memoria sobre el DataFrame. El TXT permanece intacto.

5.2 Exploración inicial: head() y describe()

Con head() se revisan las primeras cinco filas y se comprueba la estructura de la tabla. Con describe() se obtienen estadísticas como el número de valores disponibles, el promedio, la desviación estándar, los mínimos, máximos y cuartiles.

5.3 Identificación de faltantes: contarNulos(df)

La función recorre los registros con iterrows(), comprueba mediante isnull().any() si una fecha tiene por lo menos un campo vacío y guarda en un diccionario la cantidad de nulos por fecha. Además, utiliza df.isnull().sum() para calcular los faltantes por columna y después los suma para obtener el total general.

```
nulos_por_columna = df.isnull().sum()
for columna, cantidad in nulos_por_columna.items():
    print(f"{columna}: {cantidad} valores faltantes")
print(f"Total de valores nulos: {nulos_por_columna.sum()}")
```

5.4 Tratamiento de valores nulos: limpiar_datos(df)

Se crea una copia del DataFrame original mediante copy(), de manera que puedan compararse los resultados. Para cada variable numérica se calcula la media mediante mean() y se reemplazan sus NaN con fillna(). No se altera la columna FECHA.

```
for columna in columnas_numericas:
    media_columna = df_limpio[columna].mean()
    df_limpio[columna] = df_limpio[columna].fillna(media_columna)
```

5.5 Visualización y función principal

La función graficar_histogramas_y_boxplots() presenta histogramas comparativos de precipitación y evaporación, así como un boxplot de temperatura máxima. Por su parte, main() reúne la lectura del archivo, el conteo de faltantes y la limpieza. Las comparaciones y gráficas del notebook se ejecutan también en celdas independientes.

6. Resultados principales

La exploración detectó 312 fechas que tienen al menos un valor ausente. Al contar cada celda faltante por separado, el total asciende a 323 valores nulos. La diferencia ocurre porque algunas fechas tienen más de un dato incompleto.

Variable	Nulos antes	Nulos después	Media de imputación
FECHA	0	0	No aplica
PRECIP	40	0	2.22 mm
EVAP	272	0	4.88 mm
TMAX	5	0	34.47 °C
TMIN	6	0	16.66 °C
Total	323	0	—

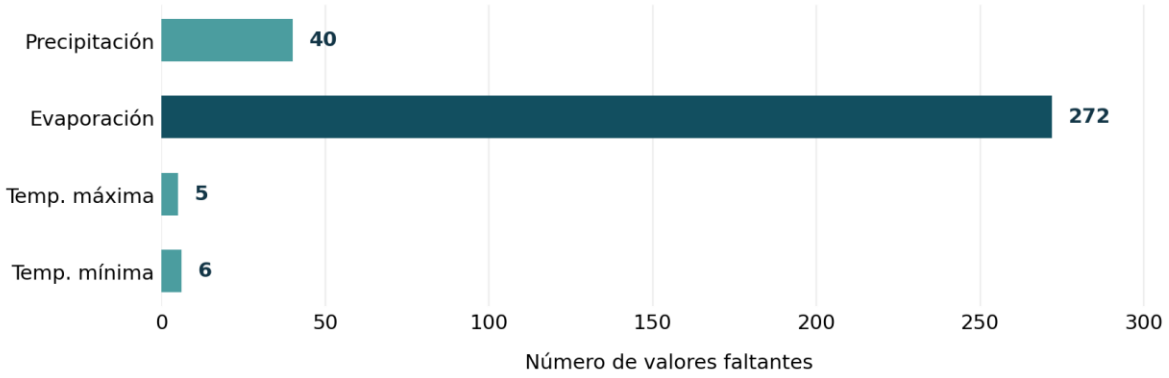


Figura 1. Cantidad de valores faltantes por variable antes de la limpieza.

Resultado destacado. La evaporación concentra 272 de los 323 valores faltantes, por lo que es la variable más afectada por el procedimiento de imputación.

7. Comparación antes y después

Después de aplicar la imputación por la media, cada variable numérica cuenta con 19,574 valores en el resumen estadístico, frente a cantidades menores en la base original. Los promedios se mantienen prácticamente iguales por la naturaleza del método empleado; sin embargo, la dispersión puede disminuir levemente.

Variable	Valores antes	Valores después	Promedio
PRECIP	19,534	19,574	2.22 mm
EVAP	19,302	19,574	4.88 mm
TMAX	19,569	19,574	34.47 °C
TMIN	19,568	19,574	16.66 °C

Los histogramas del notebook permiten observar que los registros de precipitación se concentran cerca de cero y que en evaporación se añaden observaciones en torno al promedio al completar los nulos. El boxplot de TMAX ayuda a revisar si cambian la mediana, la dispersión o los valores atípicos. Las visualizaciones cumplen una función comparativa; no permiten afirmar que se hayan recuperado las mediciones originalmente ausentes.

8. Limitaciones del método

La imputación por la media es sencilla y permite trabajar con un conjunto numérico sin celdas vacías en las variables seleccionadas. No obstante, sustituye observaciones desconocidas por un valor promedio, reduce potencialmente la variabilidad y no considera la estacionalidad, la fecha concreta ni las condiciones meteorológicas asociadas a cada registro. Por ello, los datos completados deben tratarse como estimaciones, no como mediciones originales.

9. Resumen de lo realizado

- Importamos Pandas y Matplotlib y cargamos el archivo 25001.txt.
- Ignoramos el encabezado informativo y excluimos mediante código la línea final de guiones.
- Exploramos el DataFrame con head() y describe().
- Contamos los nulos por fecha y columna: 312 fechas incompletas y 323 datos ausentes.
- Completamos los nulos en PRECIP, EVAP, TMAX y TMIN usando la media de cada columna.
- Comparamos las estadísticas y las distribuciones mediante tablas e histogramas y un boxplot.

10. Conclusiones

El proyecto permitió poner en práctica funciones, condicionales, ciclos, diccionarios y herramientas de análisis de datos con Pandas. Se construyó un procedimiento reproducible que conserva intacto el TXT original, identifica valores faltantes y genera un DataFrame completo en sus columnas numéricas. La comparación posterior permite reconocer tanto la utilidad como las limitaciones del reemplazo por la media. Como continuación del trabajo, sería pertinente evaluar métodos que tengan en cuenta la estacionalidad y los patrones temporales del clima.

Fuentes de trabajo

CONAGUA-DGE, base de datos climatológica de la estación 25001, archivo 25001.txt (emisión 06/04/2020); notebook del proyecto, TrabajoEnEquipo_con_guion.ipynb.

Anexo. Código Python utilizado

Se reproduce el código del notebook en el mismo orden de sus celdas. Las secciones explicativas anteriores describen su función dentro del proyecto.

A. Librerías

```
import pandas as pd
import matplotlib.pyplot as plt
```

B. Función de lectura del TXT

```
def cargarCSV(ruta):
    nombre_columnas = ['FECHA', 'PRECIP', 'EVAP', 'TMAX', 'TMIN']
    df = pd.read_csv(ruta,
                     sep=r'\s+',
                     skiprows=19,
                     names=nombre_columnas,
                     na_values=['Nulo'],
                     encoding='latin-1'
                    )

    # Eliminamos la última fila si contiene únicamente guiones.
    # El archivo TXT original se conserva sin cambios.
    if not df.empty and str(df.iloc[-1]['FECHA']).strip('-') == '':
        df = df.iloc[:-1].copy()

    return df
```

C. Exploración inicial

```
# Llamamos a la función y guardamos los datos en la variable 'df'
df = cargarCSV('25001.txt')

#Mostramos la tabla
print("\n--- Vista de la tabla---")
print(df.head())

#Un  describe para ver las estadística
print("\n--- Nuevo Resumen Estadístico---")
print(df.describe())
```

D. Conteo de valores nulos

```
# Contar los valores nulos por fecha y por columna
def contarNulos(df):
    print("--- Conteo de registros faltantes por fecha ---")
    faltantes = {}

    for indice, fila in df.iterrows():
        if fila.isnull().any():
            cantidad_nulos = fila.isnull().sum()
            fecha = fila['FECHA']
            faltantes[fecha] = cantidad_nulos

    print(f"Se encontraron {len(faltantes)} fechas con datos faltantes.")
    print("Ejemplo de las primeras 5 fechas:")
    for fecha in list(faltantes.keys())[:5]:
        print(f"Fecha: {fecha} | Faltantes: {faltantes[fecha]}")
```

```
    # Contamos los nulos en cada columna, incluyendo FECHA.
    print("\n--- Total de valores nulos por columna ---")
    nulos_por_columna = df.isnull().sum()
    for columna, cantidad in nulos_por_columna.items():
        print(f"{columna}: {cantidad} valores faltantes")

    # Sumamos todos los valores faltantes de la tabla.
    print(f"Total de valores nulos: {nulos_por_columna.sum()}")
```

E. Limpieza por la media

```
def limpiar_datos(df):
    print("Rellenando nulos con la media de la columna")

    df_limpio=df.copy()
    columnas_numericas=['PRECIP','EVAP','TMAX','TMIN']

    for columna in columnas_numericas:
        media_columna=df_limpio[columna].mean()
        df_limpio[columna] = df_limpio[columna].fillna(media_columna)
        print(f"Columna '{columna}': nulos reemplazados con la media -> {media_columna:.2f} ")
    return df_limpio
```

F. Gráficas comparativas

```
def graficar_histogramas_y_boxplots(df_original, df_limpio):
    """Compara la limpieza con dos histogramas y un boxplot."""

    # Histogramas de PRECIP y EVAP: muestran cómo cambia su distribución.
    for columna in ['PRECIP', 'EVAP']:
        fig, ejes = plt.subplots(1, 2, figsize=(10, 4))

        ejes[0].hist(df_original[columna].dropna(), bins=20, edgecolor='black')
        ejes[0].set_title(f'Histograma antes: {columna}')
        ejes[0].set_xlabel(columna)
        ejes[0].set_ylabel('Frecuencia')

        ejes[1].hist(df_limpio[columna].dropna(), bins=20, edgecolor='black')
        ejes[1].set_title(f'Histograma después: {columna}')
        ejes[1].set_xlabel(columna)
        ejes[1].set_ylabel('Frecuencia')

        fig.tight_layout()
        plt.show()

    # Boxplot de TMAX: permite comparar la dispersión y valores atípicos.
    plt.figure(figsize=(7, 4))
    plt.boxplot(
        [df_original['TMAX'].dropna(), df_limpio['TMAX'].dropna()],
        tick_labels=['Antes', 'Después']
    )
    plt.title('Boxplot antes y después: TMAX')
    plt.ylabel('TMAX (°C)')
    plt.show()
```

G. Comparación estadística

```
df_lleno = limpiar_datos(df)

#Mostramos la tabla DESPUÉS de llenar
print("\n--- Vista de la tabla DESPUÉS de llenar ---")
print(df_lleno.head())
```

```
#Un nuevo describe para ver como cambió la estadística
print("\n--- Nuevo Resumen Estadístico (Post-Limpieza) ---")
print(df_lleno.describe())
```

H. Ejecución de visualizaciones

```
# Comparación visual antes y después de la limpieza
print("--- Histogramas y boxplots antes y después de la limpieza ---")
graficar_histogramas_y_boxplots(df, df_lleno)
```

I. Función principal

```
def main():
    archivo='25001.txt'
    datos=cargarCSV(archivo)
    contarNulos(datos)
    datos_limpios = limpiar_datos(datos)
```

J. Inicio del programa

```
if __name__ == "__main__":
    main()
```